

The Observed-Information Program

Gene Wen · jw9263@nyu.edu

A research program playbook whose signature is an information-rate co-design running from a quantitative training objective to a power-efficient datapath, with an information-theoretic theorem on the limits of self-improvement as its companion.

Working draft, July 2026. Internal planning document with detailed execution instructions.

Executive summary

The Observed-Information Program is not nine equal projects under one slogan. It is a small-lab portfolio with one signature spine and one companion spine. The signature spine turns coded information rate into a design objective that runs from training to pruning to bit-plane execution, giving the group a differentiated position at the intersection of source coding, model compression, quantitative performance modeling, and low-power architecture. The companion spine studies when computation can change a decision at all: self-generated text alone adds no observed information, while tools, tests, verifiers, independent models, and observed futures can. The shared information ledger is the reason these projects belong in one lab; each paper, however, must stand on its own evidence.

Track	Core projects	Role	Success target
Signature spine: information-rate co-design	BackSlash, Residual-Constrained Training, bit-plane scheduling, AUGUR cost model	Main lab identity and most executable portfolio; turns coded rate into a trainable, schedulable quantity	One methods and systems paper: rate-trained models smaller, more prunable, and cheaper to execute than post-hoc baselines
Companion spine: observed-information reasoning	No-Free-Information theorem, Forward-Backward POE, directionality	High-ceiling intellectual bet; explains when self-generated computation helps and when it cannot	One ICLR or ACL paper: boundary theorem plus E1-E4 and a deployment or decoding diagnosis
Gated systems line	Admission control and AUGUR, only after simulation passes	Turns the ledger into a serving policy under tail latency	Proceed only if the saturation-knee simulation and one real vLLM confirmation are clean
Deferred / appendix	HOSVD; broad SphereMerge and FuseCorrect expansion	Useful support, not central to the next twelve months	Supporting evidence or future work unless a decisive result appears

The projects are therefore not equally active. They are labeled Core for the next twelve months, Gated to start only after a precondition is met, Support, and Deferred, and the schedule in Section 8 runs only the Core and Gated tracks. Audiences are addressed differently: machine-learning readers are led with the theorem, systems and hardware readers with the rate co-design and AUGUR, and the shared ledger is the lab identity rather than the headline of any single paper.

0. How to read this document

This playbook brings nine ongoing projects together into a single research program and gives step by step instructions to carry it out. Sections 1 to 3 state the thesis, the two spines the projects fall into, the flagship theorem, and the unifying law. Sections 4 to 6 are the operational core, organized by spine: Section 4 is the companion spine in language, Section 6 is the signature spine in the model, and Section 5 is the serving layer that bridges them, with admission on the companion side and AUGUR on the signature side. Each gives concrete

protocols, controls, expected outcomes, and go or no-go criteria, and Section 6.6 sets out the shared coding machinery. Sections 7 to 12 cover methodology, the twelve month timeline, risks, deliverables, and positioning against prior work, Section 13 assesses impact and recommends how to frame the program, Section 14 sets out where the work could become groundbreaking, and Section 15 keeps the full program map as an internal synthesis. The program leads with its signature spine, an information-rate co-design that runs from a quantitative training objective to a power-efficient datapath and draws on classical source coding, quantitative performance modeling, and low-power architecture; the theorem on the limits of self-improvement is its striking companion. The appendices fix notation, the mutual information estimation protocol, a reproducibility checklist, and a full run sheet for the pivotal experiment. Every instruction is written so a collaborator can run it without needing the source papers.

0.1 The nine projects at a glance, by priority

Project	In one line	Priority and role
BackSlash	Rate-constrained training that codes LLM weights near their true information rate	Signature, core: training-time weight coding
Residual-Constrained Training	Prune blocks by the coded rate of the residual each layer injects	Signature, core: depth-pruning signal
Bit-plane scheduling	Allocate compute and communication per bit-plane by its occupancy, MSB-first	Signature, core: information-proportional execution
AUGUR	Spec-first, uncertainty-aware LLM performance estimator	Signature, core: cost and performance model
Forward-Backward POE	Product-of-experts fusion of a forward and a backward LM for token correction	Companion, core: the theorem and its evidence
Admission control	Gate requests on a calibrated latency tail rather than a point estimate	Gated: serving instance, after simulation
FuseCorrect	Repair a pruned layer segment with a causal token-graph corrector	Deferred: intra-model instance
SphereMerge	Merge redundant layers by debiased directional geometry	Deferred: localization
HOSVD	Higher-order SVD of weight-training trajectories	Deferred: off-axis foundation

1. Thesis and the information ledger

Across all nine projects one principle keeps returning. Only observed information changes a decision. Self-generated or purely modelled information, however plausible it looks, carries no incremental value. We call the bookkeeping of this principle the information ledger. Every computation a system performs falls into one of three lines. There is information it observes, whether from the world, a tool, a sensor, or an independently trained source. There is information it generates from what it already has, which is free but inert. And there is the physical cost of moving whatever information it uses.

An efficient system therefore has three jobs, and the program is organized around them. First, locate where the real, observed information is. Second, pay only the physical cost of moving that information. Third, never spend computation treating generated information as if it were observed. The first job is a question of information availability, answered by the flagship theorem and its instances. The second is a cost question, answered by AUGUR. The third is a discipline we call the honesty budget, enforced by calibration, grounding labels, and regime maps throughout.

The bet is that this ledger is measurable rather than metaphorical, and that one inequality governs phenomena that currently look unrelated: whether a language model can correct itself, whether a serving gate should admit

a request, and whether a pruned network layer can be repaired. If the bet holds, the program is a single lens rather than nine separate papers.

1.1 The two spines

The nine projects are not a flat list. They fall into two spines that share the ledger but differ in method, maturity, and the strength of the ground the program stands on. One is the signature and one is the companion, and reading the program that way is clearer than treating it as a single undifferentiated law (Figure 1).

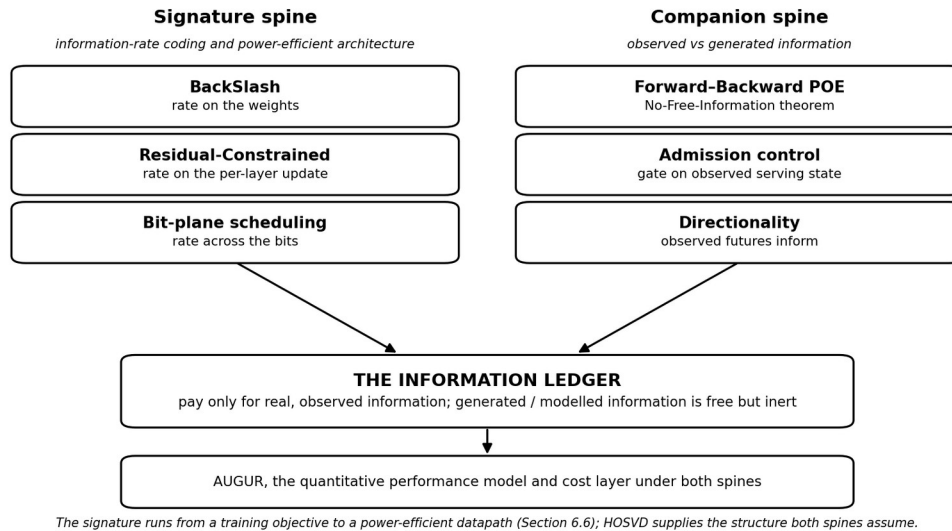


Figure 1. Two spines under one ledger. The signature spine turns the coded information rate into a design objective from training down to the datapath; the companion spine bounds what information is available to a computation at all.

The signature spine is about rate and its physical realization. It makes the coded information rate of a neural quantity a differentiable objective, then carries that objective down to the hardware. Its members are BackSlash on the weights, Residual-Constrained Training on the per-layer update, and bit-plane scheduling across the bits, all resting on established source-coding results for exp-Golomb coding of generalized-Gaussian sources and extending them to modern networks (Section 6.6), with AUGUR supplying the quantitative performance model that prices each bit. What makes it the signature is that it runs end to end, from a training loss to a power-efficient datapath (Figure 5), on an axis where classical source coding, quantitative performance modeling, and low-power architecture meet. It is concrete, already published or submitted, reusable across granularities, and it is where the program is most differentiated and least dependent on resources it does not have.

The companion spine is about availability. It asks which information can change a decision at all, and its members are the No-Free-Information theorem, admission control, and the directionality result that a backward model reading the observed future carries information no forward context can supply. Its ceiling is the highest single thing in the program, because a rigorous negative result in the self-improvement debate is cited for years, but its variance is higher and it competes on ground the large model labs also occupy. It is the striking companion to the signature, not the lead.

Presenting the program signature first and companion alongside is both the more honest reading and the stronger positioning, as Section 13 argues. The ledger is a tight statement inside each spine and a looser one across them, so leading with the co-design spine lets the program stand on the axis where it is most nearly unique, while the theorem provides the high-ceiling swing.

2. The flagship theorem (No Free Information)

2.1 Statement

Let $x_{<t}$ denote the observed context, x_t the current token, and f any deterministic function of the context. Then the conditional mutual information vanishes: $I(x_t ; f(x_{<t}) \mid x_{<t}) = 0$. In plain terms, no self-generated continuation adds any information about the current token beyond what the context already holds. This covers chain-of-thought, self-critique, best-of-N sampling from the same model, and iterative self-refinement. Stated safely, the claim is not that models cannot improve. It is that self-generated text alone adds no observed information, so improvement requires an external channel: an independent model, a tool, a test, a verifier, a selection signal, or environment feedback.

2.2 Proof sketch

Condition on $x_{<t}$. Because f is a deterministic function of $x_{<t}$, its value is fixed once $x_{<t}$ is fixed, and a constant shares zero mutual information with anything. The result follows from the data-processing inequality applied to the Markov chain $x_t, x_{<t}, f(x_{<t})$. Under stochastic decoding with temperature above zero, f is no longer deterministic, but the bound still holds in expectation over the sampling noise, because that noise is independent of x_t given $x_{<t}$. The temperature control in Section 4.2 verifies this empirically.

2.3 What breaks the bound, and why it matters

Two things break the bound, and pinning them down is where the practical payoff lives. The first is an external observation, such as a tool result, a unit-test execution, or a retrieval hit. It injects information that is not a function of $x_{<t}$, so its conditional mutual information with x_t can be positive. The second is an independently parameterized model. From the joint distribution over data and parameters, that model is not a deterministic function of $x_{<t}$, so its predictions can carry non-zero information. That non-zero quantity is exactly what Section 4.2 sets out to measure. Every recommendation in this document follows from the split between these two cases. Spend compute on observation and on independent signal, and never on self-generation.

2.4 Corollaries to state explicitly in the paper

Corollary 1, the self-improvement ceiling. Without an external channel, no amount of self-generated reasoning improves local token correctness. This is a clean negative result in the open debate over whether models can improve themselves. Corollary 2, directionality is irreducible. A backward model that reads the observed future supplies information no amount of additional forward context can supply, because the future tokens are observations rather than functions of the past. Corollary 3, capacity is not the scarce resource. Since the gain comes from directionality rather than model size, a small backward model can correct an arbitrarily large forward model. Section 4.3 tests this claim.

2.5 A worked example (why this is not obvious)

Consider a model that has produced a chain-of-thought and is now asked to critique its own answer. The critique feels informative, since it is new text the model had not written before. The theorem says it is not. The critique is a deterministic function of the same context that produced the answer, so once we condition on that context it is fixed and shares zero information with the correct token. What feels like new evidence is only surface novelty. Now run the generated code against a test suite. The pass or fail bit is not computable from the context alone, because it depends on the external interpreter, so it carries real information and can change the correction. The program is the disciplined pursuit of that difference.

3. The unifying law and its nine instances

The same inequality governs every project. Each one either locates observed information, prices its movement, or refuses to fabricate it. The table below states, for each project, the observed information it exploits and the generated or modelled thing it must not trust. The full program map is kept as an internal synthesis at the end of the document (Figure 6) rather than as the opening claim, so a reviewer meets the evidence of each spine before the unifying picture.

Project	Observed information it exploits	Generated or modelled thing it must
---------	----------------------------------	-------------------------------------

		not trust
Forward-Backward POE	The observed future tokens $x_{>t}$, or tool and test results	A self-predicted future from the same model (~ 0 nat)
Admission control	The observed serving state (queue depth at admission)	An offline latency prior blind to the queueing tail
AUGUR	Measured calibration rows and hardware specifications	An unlabeled extrapolation presented as if measured
FuseCorrect	Cross-token structure accessible in the boundary window	A pointwise fit pretending to recreate token mixing
SphereMerge	The directional component of the representation	The magnitude or norm, which is inert for the task
BackSlash	The true information rate of the weights, a heavy-tailed generalized-Gaussian source	The nominal parameter count or bit-width as a proxy for information
Residual-Constrained Training	The residual update a layer injects, and its coded rate	Weight norms or block cosine similarity, dominated by the identity path
Bit-plane scheduling	The occupancy of each bit-plane, high near the MSB, dense near the LSB	Uniform per-bit cost, as if every plane carried equal information
HOSVD	The low-rank subspace shared across trajectories	Off-axis directions with no cross-task structure

4. Companion spine: language and correction (Forward-Backward POE)

This is the applied face of the companion spine, and it owns the theorem and its sharpest evidence. It has four workstreams: generalize the theorem, run the independent-training experiment that pins down the break condition, establish scale invariance, and demonstrate the law in a real deployment where the future is naturally observed.

4.1 Generalize the theorem

Step 1. Rewrite the current Section 4 result, which is stated only for predicted tokens, in terms of an arbitrary deterministic function f of $x_{<t}$ (Section 2.1). Step 2. Enumerate the important instances of f and state each as a named corollary: chain-of-thought traces, self-critique, Constitutional-AI-style revisions, best-of-N from the same model, and iterative self-refinement. Step 3. Prove the stochastic-decoding extension in expectation form and cite the temperature control (4.2) as its empirical confirmation. Step 4. Write the two break conditions as a positive design rule. Information helps only when it is an external observation or an independently parameterized source. Deliverable: a self-contained theorem section with three corollaries.

4.2 Independent-training experiment E1-E4

Purpose. Locate the exact edge of the bound. A same-model prediction carries zero information (E1), the true observed future is the information ceiling (E3, the oracle), and the POE-trained backward model is the method (E4). The critical unknown is E2, how much a genuinely independent model contributes, because E4 minus E2 is the single number that shows directionality beats mere model diversity.

Setup. Train two GPT-2 124M models with identical architecture and data (WikiText-103) but different seeds for both initialization and data order. Model A uses seed 42, model B uses seed 137, each trained for 100k steps with matched hyperparameters. Confirm genuine independence by checking that the held-out logits correlation falls between 0.70 and 0.90. If it sits above that range the models are too correlated, so re-seed; if it falls below, re-check the training.

Measurement. For each test position t , form the fused log-probability $\log p_f(\cdot|x_{<t}) + \alpha \cdot \log p_{\text{pred}}(\cdot|\text{future})$ with $\alpha = 0.9$, and record the KL gain over the forward-only distribution at the true token. Average over positions to get the mutual information in nats. Run the four conditions in the matrix below.

Expt	Predictor of the future	Expected MI	Purpose
E1	Model A predicts its own future	≤ 0.0019 nat	Reproduce the DPI zero baseline
E2	Model B (independent) predicts	$> 0.0019, < 0.368$ nat	Quantify model-diversity information
E3	True observed future $x_{>t}$ (Oracle)	≈ 0.368 nat	Information ceiling from observation
E4	POE-trained backward model	≈ 0.662 nat	The method's realized gain

Interpretation matrix. Read the four comparisons below before writing any conclusion. The E4 minus E2 gap is the paper's single most important number.

Key comparison	Result meaning	Action
$E2 - E1 \approx 0$	Model diversity adds no information	Attribute the gain to directionality (strongest)
$E2 - E1 > 0$	Diversity carries some information	Quantify its share, still compare against E4
$E4 - E2 \gg 0$	Directionality beats diversity	Core claim holds, lead with it
$E4 - E2 \approx 0$	Gain may be mostly diversity	Re-examine, reframe honestly if it replicates
$E3 - E2 \gg 0$	Diversity cannot replace observation	Observed future is irreplaceable

Controls, run all three. Temperature sweep: repeat E1 at T in $\{0.0, 0.5, 1.0, 1.5\}$ and confirm the mutual information stays near zero, which closes the objection that stochastic decoding evades the bound. Independent-model scale sweep: run E2 with predictors of 45M, 124M, 355M, and 1.5B to separate independence from capacity. Row-shuffle sanity check: randomly permute model B's logits across positions and confirm the estimate collapses to about zero, which shows the estimator is not leaking. Decision rule: if E2 minus E1 is about zero the backward gain is purely directional, the strongest outcome; if E4 minus E2 stays large, directionality dominates diversity and the core claim holds.

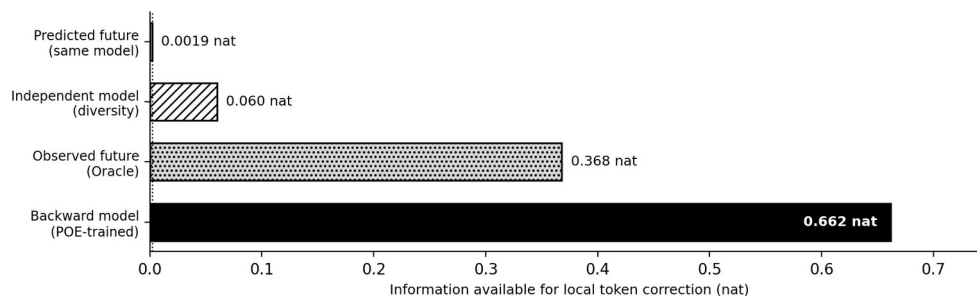


Figure 2. Measured spine of the theorem, per error position. The independent-model bar is what E2 will pin down, shown here as an illustrative mid-range value.

4.3 Scale-invariance experiment (7B/13B forward)

Purpose. Show that a fixed micro backward model corrects an arbitrarily large forward model. Step 1. Hold the backward model fixed at 45M. Step 2. Sweep the forward model across 124M, 355M, 774M, 1.5B, then 7B and

13B, recording the retained fraction of the backward advantage at each size. It already sits near 80% from 124M to 1.5B, and about 101% at 45M against 124M internally. Step 3. Fit the retention curve and report whether it plateaus. Go criterion: retention stays above about 70% at 7B and 13B. The headline if it holds: directionality, not capacity, is the scarce resource, and a tiny fixed corrector becomes a strong efficiency argument for edge deployment.

4.4 Deployment with a naturally observed future (code generation)

Purpose. Turn the theory into a systems result by finding a setting where the observed future comes for free. The primary target is code generation on HumanEval, where executing the unit tests is a genuine external observation that breaks the bound. Step 1. Generate a candidate completion. Step 2. Execute the tests and feed the pass or fail outcome and any error traces to the corrector as the observed signal. Step 3. Run end to end generate then correct, and report accuracy against a no-correction baseline and against same-model self-refinement, which the theorem predicts should not help. Secondary targets are tool-call returns and verifier checks on multi-step math (GSM8K). Deliverable: one external task-accuracy number that turns the impossibility theorem into a working system.

5. Serving: the gated line and the cost model (Admission, AUGUR)

This section spans both spines at the serving layer. Admission control belongs to the companion spine and the gated systems line, since it gates on observed serving state; AUGUR belongs to the signature spine as its quantitative performance and cost model. Together they show the same inequality in systems form and supply the cost side of the ledger.

5.1 Load-aware admission gate

Motivation. The current admission result is validated only in the easy regime: per-token decode, 0/1 cost, static load, homogeneous workload. There a calibrated scalar margin equals the full predictive distribution. The open problem is load. Above saturation, end to end latency is 90 to 95% queue wait, and every offline-calibrated gate is blind to it. Step 1. Build a continuous-batching serving loop driven by measured decode-versus-batch curves under Poisson and bursty (MMPP) arrivals on RTX 4090 and H200. Step 2. Sweep the arrival rate through the saturation knee, expected near 11 to 13 requests per second, and confirm the knee where violation and queue-wait fraction jump together. Step 3. Compare offline gates (point, margin, distribution) against a gate that conditions on the observed queue depth at admission. Expected outcome, stated as the law: offline priors admit about 100% and violate 89 to 97% above the knee, and only the observed-state gate holds. Step 4. Confirm the knee location and tail magnitude against a real vLLM run. Deliverable: a load-aware gate that proves the inequality in practice. Observed serving state carries the tail information, and modelled priors do not.

5.2 AUGUR as the economics layer

Role. AUGUR prices the movement of each bit and labels every number as measured, partially measured, or modelled. This is the honesty budget in numerical form. Step 1. Wire AUGUR's sub-millisecond per-operation estimate in as the cost term, so the program's decision rule becomes: act only when value exceeds cost, where value is the observed-information gain and cost is AUGUR's bandwidth-bound price. Step 2. Close the two headline AUGUR gaps that gate its credibility. The first is the cold-start benchmark against spec-only simulators (LLMCompass, LIFE, LIMINAL, LLM-Viewer) on a common corpus. The second is the set of modifier rows that are modelled only today, namely KV-cache compression, speculative decoding, and continuous batching. Step 3. Keep the bandwidth-hull guard. When a target falls outside the calibration hull, fall back to a physics estimate with wide intervals rather than an overconfident correction. Deliverable: AUGUR positioned as the economics that makes the program operational, not as a separate project.

6. Signature spine: information-rate co-design (with deferred structure work)

This is the core of the signature spine. The rate co-design of Sections 6.3 to 6.6, BackSlash on the weights, Residual-Constrained Training on the updates, bit-plane scheduling across the bits, and the shared machinery

beneath them, is the lab identity. FuseCorrect and SphereMerge, in Sections 6.1 and 6.2, are deferred support kept here for completeness. Together the section shows the availability bound inside a network: what cross-token structure a corrector can access, where the task information lives, how much information the weights and updates truly carry, and how it is distributed across the bits.

6.1 FuseCorrect (deferred support)

Claim to establish. A deleted transformer segment removes cross-token routing that a pointwise corrector cannot recreate by construction. Only an operator with access to cross-token structure can repair it. Step 1. Delete a contiguous K-layer segment, for example 7 layers of Llama-3.1-8B, chosen by a baseline-compatible selector. Step 2. Insert the two-branch corrector. An affine branch with closed-form initialization restores channel alignment, then a causal token-graph branch builds a sparse graph over a local causal window from learned feature distances and runs a few unrolled graph-diffusion steps. Step 3. Enforce the matched-budget protocol. Every method shares the deleted segment, the calibration data, and the evaluation suite, compared at equal parameter budget against training-free LinearPatch, an LLM-Streamline-style MLP replacement, and the affine-only ablation. Step 4. Quantify the availability floor. Show that the best pointwise map still leaves about 49% of the delete-only boundary error, and that about 42% of the segment's response energy is genuinely cross-token. Deliverables: fill in the headline perplexity and multiple-choice results and the affine-only versus full delta, and state the result as an availability bound.

6.2 SphereMerge (deferred support)

Claim to establish. Task information lives in the direction of the hidden state, not its norm, and the normalized sphere is contaminated by a few massive-activation components that must be suppressed before scoring redundancy. Step 1. Reproduce the probe result that a direction-only linear probe matches full-activation accuracy, about 80.06% against 80.01%, while a norm-only probe collapses to chance at about 53.44%. Step 2. Confirm that after L2 normalization a single massive-activation principal component explains about 81.2% of variance on LLaMA-3-8B. Step 3. Score redundancy with Debiased Spherical Information Alignment. Project out the top-k principal directions, normalize, and score soft assignments to shared anchors under a normalized-mutual-information criterion with one temperature. Step 4. Fuse with magnitude-preserving weighting and fold in the closed-form Fusion-aware Compensation scalar, so inference cost is unchanged. Report the 42-times scoring speedup over the diffusion-kernel baseline and the clean composition with 4-bit quantization. The framing: pruning removes the inert, information-free axes.

6.3 BackSlash (signature)

Claim to establish. A model's weights carry far less information than their bit-width suggests. Trained parameters follow a generalized-Gaussian distribution with shape below 2, so their true rate is a small fraction of the nominal size, and rate-constrained training can drive the model to that floor without losing accuracy. Step 1. Fit the generalized-Gaussian shape parameter to each layer's weights and confirm it sits below 2, for example about 1.36 for BERT, 1.54 for GPT-2, 1.26 for Llama-3, and 0.85 for DeepSeek. Step 2. Define the discretized generalized-Gaussian rate $R(\theta)$ as the mean of $|\theta_i|$ raised to the power of the shape parameter, estimating the shape per batch from the ratio $E[\theta^2]$ over $E[|\theta|]^2$. Step 3. Train with the rate-distortion objective $J = D + \lambda R$, using soft gradient clipping (add a small ϵ inside the power) to stop the gradient from blowing up when the shape drops below 1. Step 4. Entropy-code the trained weights with exp-Golomb codes at $k = 0$ after ranking parameter values by frequency, which lands within a few points of the entropy limit and needs no per-model table. Expected outcome: 75 to 90% size reduction with roughly 1 to 2% accuracy change, and models that stay accurate under pruning to 80 to 90% where ordinary training fails past 60%. The framing: measure and pay for the weights' real information rate rather than their nominal precision. As a bonus, the extra sparsity enlarges the headroom that the FuseCorrect and SphereMerge instances exploit.

6.4 Residual-Constrained Training (signature)

Claim to establish. The right pruning signal for a Transformer block is the residual it injects, not its weight norm or its input-output similarity. Writing a block as $h_{\ell+1} = T_{\ell}(h_{\ell}) = h_{\ell} + r_{\ell}$, the residual $r_{\ell} = T_{\ell}(h_{\ell}) - h_{\ell}$ is exactly the additive contribution of the layer, so its coded size measures how much new information the layer adds to the forward stream. Step 1. Collect per-layer residuals and confirm the two-part structure that makes them a good signal: spatially heterogeneous across depth, so layers differ, but temporally homogeneous across batches, so the statistic is stable and reproducible. Step 2. Fit a generalized-Gaussian to each layer's residual and verify the shape sits below 2 (super-Gaussian, low entropy), which separates layers far more clearly than weights (near-Gaussian) or activations (uniformly sharp), both of which barely vary across depth. Step 3. Turn the distribution into a differentiable per-layer rate R_{ℓ} using the signed exp-Golomb code-length surrogate, the same coding tool as BackSlash, with a sign-balance and expectation-matched relaxation so it is differentiable in the residual. Step 4. Train with R_{ℓ} as the structure signal: a layer whose residual rate is low injects little information and is a prune or merge candidate. Expected outcome: a block-level criterion that is absolute and additively composable, unlike the scale-invariant cosine similarity, and that exposes clearer layer-wise heterogeneity. The framing: measure the information each layer actually injects, its observed residual rate, and drop the layers that inject almost none. This is the depth-pruning counterpart to BackSlash's weight coding, and it produces the selection that FuseCorrect and SphereMerge then act on.

6.5 Bit-plane scheduling (signature)

Claim to establish. Bit significance is an information-proportional scheduling axis, not just a precision knob. A weight decomposes as a sum over binary planes, $W = \sum_b 2^b \cdot W_b$, so $W \cdot x = \sum_b 2^b (W_b \cdot x)$, and each plane's product is an independent binary matmul that recombines by shift-and-add. For heavy-tailed generalized-Gaussian weights the high-significance planes are extremely sparse while the low planes are dense and near-random, so a plane's occupancy is a direct read-out of the information it carries. The scheduling rule follows: give the sparse, information-rich planes dedicated sparse kernels and send them first, and drop or approximate the dense, inert low planes. Figure 4 shows the occupancy falling off toward the MSB for a generalized-Gaussian source.

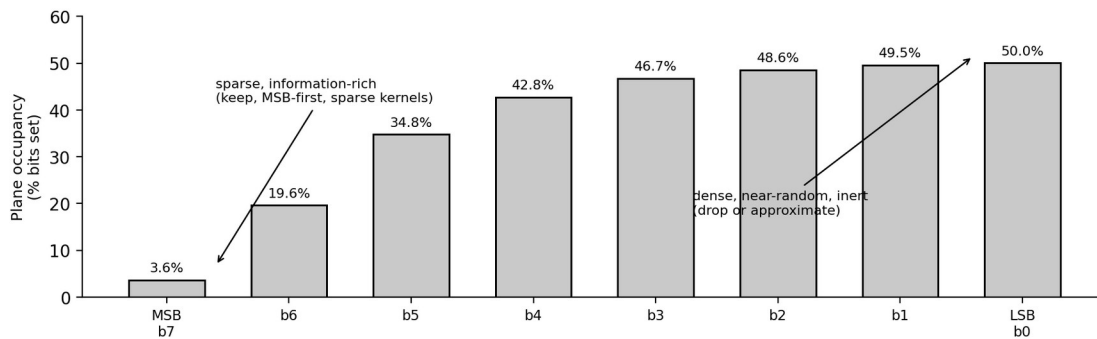


Figure 4. Bit-plane occupancy for a generalized-Gaussian weight source (shape 1.3, 8-bit magnitude). The MSB plane is about 4% occupied and information-rich; the LSB plane sits near 50% and is essentially noise.

Instructions. Step 1. Fit the plane-occupancy profile for a BackSlash-trained model and confirm the MSB planes are single-digit-percent occupied. Step 2. Route each plane by occupancy: sparse kernels and priority communication for high planes, truncation or low-precision approximation for the tail. Step 3. Add the anytime path: stream planes MSB-first and stop once the admission gate's confidence or the SLO is met, which reuses the calibrated-tail machinery of Section 5. Step 4. Price each plane's transfer with AUGUR so the keep-or-drop choice per plane is a value-minus-cost decision. Positioning caveat, to preempt the obvious objection. The plane decomposition itself already exists in any-precision and binary-coded quantization, and bit-serial accelerators already skip zero bits, so the novel claim must be the scheduling axis: per-plane compute and communication proportional to occupancy, MSB-first anytime, and priced by the cost layer. Representation matters as well; two's-complement wastes the high planes on sign extension, and BackSlash's frequency-index value mapping changes what a plane means, so the profile is measured on the representation actually deployed.

Arithmetic and hardware. The per-plane kernel is multiplier-free. A single bit-plane against a binarized (± 1) activation reduces the multiply to an XNOR and the place value across planes to a shift, which is exactly the

XNOR-and-popcount datapath of binary-network accelerators. The one operation that is not free is the accumulate: summing the per-lane XNOR results is a popcount, and popcount is addition, so it carries. XOR is only the carry-free sum bit of a full adder. This matters because the carry is how information flows from the low planes up to the high planes, and since the low planes are near-random and inert, their carries are mostly noise. A carry-save or carry-truncated accumulation that refuses to propagate low-plane carries therefore loses almost no information while decoupling the planes, which is what makes per-plane parallelism and MSB-first anytime execution legal. Plane independence is recovered exactly to the degree the low planes are inert. One clarification keeps the claim honest: the multiplier-free path here is binary-network XNOR and popcount, not the genuinely carry-free XOR-and-shift arithmetic of GF(2) codes such as CRCs or carry-less multiplication, which is a different operation and not the neural-network inner product.

6.6 Shared machinery: the generalized-Gaussian rate primitive

The three coding-spine instances are not merely thematically similar; they run on one primitive. Trained neural quantities are heavy-tailed, so a generalized-Gaussian with shape below 2 models them well, and a signed exp-Golomb code is near-optimal and robust for such sources while needing no per-model table. Making that coded rate a differentiable objective is the reusable move, and the only thing that changes between instances is the quantity it is applied to. The table below states the pattern.

Instance	Quantity coded	What the rate buys
BackSlash	The weights	75–90% smaller models, more prunable
Residual-Constrained	The per-layer update r_ℓ	A block-level pruning and merging signal
Bit-plane scheduling	The bits of the weights	Compute and communication by plane occupancy

Because the primitive is the same, the instances compose into a single chain from the training objective down to the datapath, shown in Figure 5. Rate-constrained training drives the weights to a low-information, heavy-tailed distribution; that distribution entropy-codes near the limit with a cheap exp-Golomb decoder; the same sparsity makes the model prunable to 80 to 90% and leaves the high bit-planes nearly empty; and empty high planes are exactly what a multiplier-free XNOR-and-popcount datapath wants. This end-to-end co-design, from loss function to silicon, is the coding spine’s most concrete claim on impact.

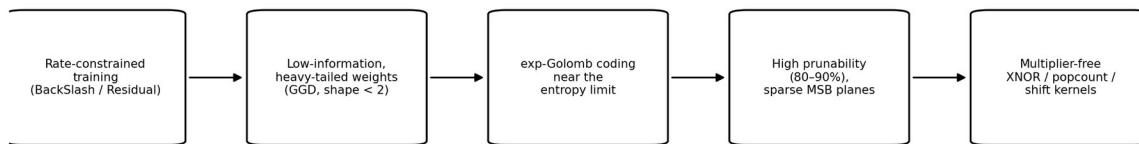


Figure 5. The coding-spine co-design chain, from a training objective to a multiplier-free datapath, all resting on the generalized-Gaussian rate primitive.

One honesty note. The primitive rests on the empirical observation that these quantities really are super-Gaussian, which holds for the weights and residuals measured so far but must be checked on the representation actually deployed, since two’s-complement layout and value-mapping change the picture. The exp-Golomb near-optimality result is for quantized generalized-Gaussian sources specifically, so a mismatch in the source model is the failure mode to watch.

7. Cross-cutting methodology: the honesty budget

The program’s second signature, after the law itself, is that every exploitation comes with an explicit account of its support. Enforce this everywhere. Label every reported number as measured, partially measured, or modelled. Attach calibrated intervals to every predictive claim and report empirical coverage against nominal

levels. Publish a regime map for each decision that shows where a cheap statistic suffices and where the full distribution is required. Use one shared mutual-information estimator across all domains (Appendix B) so the language, serving, and structure results are numerically comparable. This discipline is what makes the thesis scientific rather than promotional, and it is the part reviewers will trust.

8. Milestones and twelve-month timeline

The plan front-loads the theorem and the two lowest-risk experiments, independent training and FuseCorrect results, runs the systems and scaling work through the middle, and reserves the last quarter for the deployment result and writing. Figure 3 shows the schedule. The go or no-go gates are: theorem generalized and E1–E4 measured by M4; scale invariance and the load-aware knee reproduced by M7; deployment accuracy by M10.

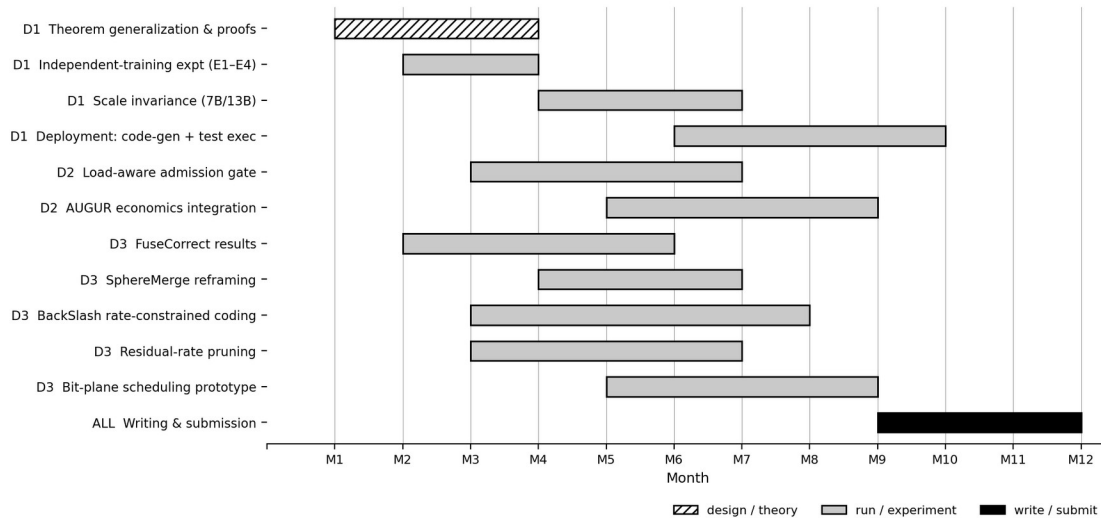


Figure 3. Twelve-month schedule across the three domains. Design and theory, experiment, and writing phases are distinguished by fill.

Read as phases for a group of two professors and fewer than ten students, the same schedule is a sequence of five gated stages. Only the Core and Gated tracks appear; deferred work is not scheduled.

Phase	Months	Owners	Work	Go / no-go
1	M1–M3	1–2 students + PI	BackSlash reproduction, residual-rate profiling, initial bit-plane occupancy on one model family; in parallel, theorem write-up and E1/E4 reproduction	Continue only if the rate signal is stable and the theorem measurements reproduce
2	M3–M6	2–3 students	Rate-constrained training and residual-rate pruning paper; add strong post-training quantization and pruning baselines; start E2 independent model and row-shuffle controls	Submit the rate paper if it beats strong post-hoc baselines on size, accuracy, and prunability
3	M5–M8	1–2 students	Bit-plane scheduling prototype and AUGUR pricing; minimal kernel or simulation showing an MSB-first, occupancy-aware advantage	Continue only if measured plane occupancy survives the deployed representation, not just simulated weights
4	M6–M10	1–2 students	Observed-information paper: E1–E4, scale sweep, verifier diagnosis, and	Submit only if E4 minus E2 stays large or the

			one deployment task with tests or tool returns	failure mode is itself a clean boundary result
5	M9-M12	All	Write the two papers, rate co-design flagship and observed-information companion; admission control proceeds only if the saturation-knee experiment is clean	No new project starts unless one of the two main papers is submitted

9. Risks and mitigations

Risk	Why it threatens the program	Mitigation
E2 $\approx$ Oracle (diversity explains the gain)	Would collapse the directionality claim into ordinary ensembling	Row-shuffle and scale sweeps to detect leakage; if real, reframe as a diversity result and report it honestly
Scale invariance fails above 1.5B	Removes the strongest efficiency headline	Report the retention curve as measured; fall back to the fixed-corrector claim at the scales where it holds
No clean deployment with observed future	Theory stays internal, weaker systems contribution	Prioritize code-gen with test execution; tool-returns and math verifiers as backups
AUGUR cross-family transfer stays weak	Limits the cost layer to near-distribution targets	Use ranking with wide intervals off-support; bandwidth-hull guard; scope claims to the interior
FuseCorrect graph diffusion underperforms	Weakens the intra-model availability instance	Matched-budget protocol still yields an informative floor result even if gains are modest
Reviewers see a forced grand unification	Undercuts the program framing	Lead with the signature spine and the theorem as two narrow papers; keep the nine-instance map as a late synthesis; mark HOSVD, SphereMerge, and FuseCorrect as deferred support

10. Deliverables, success metrics, and venues

Success is three deliverables, not nine, and the flagship is a single sharp claim gated on real-model validation, because that is what separates it from an efficiency lab that reports only simulated accounting.

Deliverable	Claim	Minimum evidence	Venue
D1: Rate co-design flagship	A model trained under a rate objective is at once smaller, more prunable, and cheaper to execute than the same model compressed after training	BackSlash and residual-rate pruning against strong post-hoc baselines, validated on at least one real 7B or larger model, with actual cycle and energy accounting for the datapath; bit-plane occupancy as a supporting section	ICML / NeurIPS / MLSys

D2: Observed-information companion	Self-generated computation helps only when an external or independent channel injects information	Generalized theorem, E1-E4, E4 minus E2, row-shuffle nulls, temperature controls, and one deployment task or verifier diagnostic	ICLR / ACL / EMNLP
D3: Gated serving paper	Observed queue state beats offline priors above the saturation knee	Simulation plus one vLLM confirmation; offline gates fail while the observed-state gate holds	MLSys / systems venue

Two of these, the rate co-design flagship and the observed-information companion, are the papers the lab is judged on. The gated serving paper follows only if its simulation is clean. No grand unification paper is attempted first: the lab identity can be broad, but the papers must be narrow.

11. Positioning against prior work

Language and self-improvement. A large body of empirical work debates whether LLMs can self-correct their reasoning, with mixed and often negative findings but no unifying account. The generalized theorem supplies that account. Without an external channel, self-generated feedback carries zero incremental local information, so any method that appears to work must be smuggling in an external signal such as a task reward, a verifier, or a tool. Framed this way, a scattered debate becomes a clean impossibility result together with a recipe, and that is the contribution reviewers will remember.

Serving and estimation. AUGUR sits among performance tools such as Vidur, MIST/HERMES, LLMCompass, and ALA, each strong on one slice. Its distinct position is spec-first coverage with calibrated uncertainty at sub-millisecond cost, which is exactly what an admission gate needs. The admission work is deliberately downstream of estimation. It consumes the calibrated tail and shows that only observed serving state closes the gap above saturation, something no offline estimator can do. The program presents the estimator and the policy as one loop rather than two isolated results.

Compression and structure. Depth-compression methods such as ShortGPT, LLM-Streamline, SLEB, Shortened-LLaMA, and MKA mostly study which layers to remove. FuseCorrect and SphereMerge reframe the field around information availability: what cross-token structure a corrector can access, and where in the representation the task information actually lives. BackSlash extends the same view to training itself. Because trained weights are a low-information, heavy-tailed source, rate-constrained training codes them near their true rate and leaves the model far more prunable, which the other structure instances then exploit. Residual-Constrained Training sharpens the selection side of the same story: it scores a block by the coded rate of the residual it injects, an absolute and additively composable signal, rather than by a weight norm or the scale-invariant input-output cosine that dominate prior depth-pruning work. Bit-plane scheduling pushes the same idea down to the bit: any-precision quantization and bit-serial accelerators already decompose weights into planes and skip zero bits, but the program's claim is the scheduling axis on top of that, where per-plane compute and communication track occupancy and the most significant planes are streamed first. Together these are a lens rather than a marginal metric gain, and it is what ties the compression work to the language and serving instances under one inequality.

12. Compute budget and resourcing

The program is staged so the highest-impact result, the generalized theorem and its E1-E4 spine, is also the cheapest to produce, and the expensive scale work comes only after the core claim is secure. The table below gives an order-of-magnitude budget per workstream. It assumes a small shared cluster and treats the two independent trainings as parallelizable. Read it as a sequencing guide rather than a procurement document. Fund Domain 1 first, and gate the rest on its outcomes.

Workstream	Approx. compute	Note
------------	-----------------	------

Theorem generalization	Negligible (analysis)	Paper and proof work, no training
E1-E4 independent training	2 GPU-trainings $\times$ 100k steps	3–5 days wall-clock in parallel
Scale invariance 7B/13B	Inference-only sweeps	Fixed 45M corrector, forward models frozen
Code-gen deployment	Generation + test execution	Compute-light, the interpreter is the observation
Load-aware admission gate	Serving-loop simulation + one vLLM run	Bulk is simulation, one real confirmation run
AUGUR economics + cold-start	Calibration + benchmark rows	Adds SGLang and modifier rows, no large training
FuseCorrect / SphereMerge	Corrector training + scoring	Small correctors, scoring is 42 $\times$ cheaper than MKA
BackSlash rate-constrained training	One training run with a rate term	Per-batch shape estimate adds minor overhead
Residual-rate pruning	Residual profiling + rate-constrained finetune	Reuses the exp-Golomb coding; light overhead
Bit-plane scheduling prototype	Kernel + occupancy profiling	Builds on BackSlash weights; no extra training

Two resourcing principles keep the plan robust. First, no expensive workstream starts before its cheap precondition is met. The 7B/13B sweep waits on the retention curve holding at 1.5B, and the load-aware gate waits on the saturation-knee simulation. Second, every workstream shares one mutual-information estimator and one honesty-budget labeling convention (Appendix B), so results compose without re-tooling and a reviewer can trace any number back to a single logged configuration.

13. Impact and framing

The program has two flagships, and which one leads is a positioning choice with real consequences. The signature flagship is the information-rate co-design of Section 6.6: making the coded rate a first-class trainable objective across weights, updates, and bits, and carrying it from a training loss to a power-efficient datapath. It has a lower ceiling than the theorem but a higher expected value, because it is concrete, already published or submitted, reusable across granularities, comes with hard efficiency numbers, and sits on an axis that very few groups occupy, where classical source coding, quantitative performance modeling, and low-power architecture meet. Making the coded rate a first-class trainable objective at every granularity is itself distinctive, since rate-distortion in network compression is almost always applied after training rather than built into it, and carrying it to silicon is rarer still. The companion flagship is the No-Free-Information theorem: the highest ceiling in the program and the piece that could make the work important rather than merely useful, but also the higher-variance bet, and the one that competes on ground the large model labs also work.

The two flagships are complementary in risk. The signature de-risks the companion: if the theorem never fully lands, the co-design still stands as a citable, shippable contribution with a working objective-to-silicon story. That is why the program’s expected impact is higher, and its variance lower, than a single-thesis reading would give.

Impact also depends on who adopts what, and the two spines address different audiences. The table maps each to the instances it is most likely to use.

Audience	What they want	Instances that fit
Frontier research labs	The self-improvement question, serving under SLOs	Theorem, admission, directionality
Inference and serving	Cost-aware routing and admission at	AUGUR, admission control

infra	scale	
GPU incumbent	Pre-silicon estimation, efficient serving stack	AUGUR, admission, the coding spine
Challenger silicon	A spec-first, cross-vendor estimator	AUGUR
Edge and on-device	Small, prunable, cheap-to-decode models	BackSlash, Residual, bit-plane

The single most important framing decision is which flagship leads. The recommendation is to lead with the signature, the information-rate co-design, because it is the axis on which the program is most nearly unique and least dependent on resources it does not have, and to present the theorem as the striking companion rather than the headline. This reverses how a purely machine-learning-trained group would pitch the work, and the reversal is the point: it plays to a rare combination of source coding, quantitative performance modeling, and power-efficient architecture instead of competing head-to-head where that combination gives no edge. Leading with the co-design does not retire the theorem; it remains the high-ceiling swing of Section 14. The shared ledger is the reason the two belong in one program, not a load-bearing claim that must carry the whole.

14. Groundbreaking potentials

It is worth stating plainly where this work could become groundbreaking rather than merely strong, and what would have to be true in each case. Groundbreaking potential is a portfolio, not a single claim, and this program has an unusually good risk structure: one very high ceiling theory bet, one memorable empirical bet, one low-variance methods bet, and a longshot. The four vectors below are ranked by ceiling, and each is stated with its precondition and honest odds so the program is not oversold.

14.1 A predictive boundary theory for self-improvement

The groundbreaking version of the theorem is not that models cannot self-improve, which is too strong and empirically false, since reinforcement learning with real verifiers and self-play plainly work. It is a clean theory of exactly when self-generated computation helps: it helps only to the extent it carries an external channel, a verifier, a tool result, a game outcome, or ground-truth labels, and its gain is bounded by that channel's information. That would turn a mess of contradictory results into a principle. The sharpest application is test-time compute: extended reasoning is self-generated, so the theorem predicts its gains cannot come from the generation itself but from an implicit external channel of search, verification, or selection. Demonstrating that reasoning gains vanish when the selection channel is removed and generation is held fixed would be a contrarian, high-visibility account of why reasoning works. Precondition: the theorem must extend cleanly from single-token correction to the multi-step trajectory setting, and the empirics must show the predicted boundary. Odds of a strong result: medium. Ceiling: very large.

14.2 Directionality as the scarce resource

The claim that a small fixed corrector reading an observed future can supervise the local correctness of an arbitrarily large model, so that directional observation rather than parameter count is the scarce resource for correctness. If the retention seen to 1.5B holds at 13B and survives a real deployment such as code with test execution, that is a surprising, quotable result cutting against a pure scaling view, and it offers cheap reliability for agents. Precondition: scale invariance must actually hold at large scale, which is an extrapolation today, and a setting with a naturally observed future is required. Odds: medium-low. Ceiling: large.

14.3 Rate as a first-class training objective

BackSlash, Residual-Constrained Training, and bit-plane scheduling are three instances of one move: make the coded information rate of a neural quantity a differentiable objective. The groundbreaking version establishes rate-distortion in the training loop as a general recipe applied everywhere, to weights, activations, gradients, and the KV cache, not a niche compression trick. The concrete win that would earn it is the co-design chain of

Section 6.6 producing a clear state-of-the-art result on real hardware at equal accuracy, cheaper because the model is at once entropy-coded, prunable, and bit-plane-sparse in a way post-hoc methods cannot match. Precondition: it must beat strong post-training quantization and pruning by a clear margin on real accelerators, and the every-granularity breadth must generalize past weights and residuals. Odds of a strong, adopted method: high. Ceiling: medium to large. This is the lowest-variance vector, most likely to yield real impact and least likely to be called revolutionary.

14.4 The ledger as a cross-domain invariant

Today the information ledger organizes; to be groundbreaking it must predict. The version that would make it a law is a single quantity, the external information injected, that quantitatively predicts the achievable improvement of a self-referential process across genuinely different domains using the same estimator, whether the process is a token corrector, a serving admission loop, or a training loop. Being able to compute a scheme's ceiling in advance from one number, in two unrelated domains, would be a real invariant, and cross-domain invariants are how a framework becomes a science. Precondition: the same estimator must make a non-trivial, confirmed quantitative prediction in at least two different domains. Odds: low. Ceiling: the largest of the four.

Vector	What must be true to land	Odds and ceiling
Self-improvement boundary	Theorem extends to multi-step trajectories; boundary confirmed	Medium odds, very large ceiling
Directionality beats capacity	Scale invariance holds at 13B; a naturally observed future exists	Medium-low odds, large ceiling
Rate as training objective	Beats post-training quant and pruning on real hardware; breadth generalizes	High odds, medium-large ceiling
Ledger as invariant	One estimator predicts outcomes in two unrelated domains	Low odds, largest ceiling

The shape of the portfolio is clear. Expected value is carried by the rate objective, which will probably produce adopted results. Visibility is carried by the self-improvement boundary and the directionality result, which are the field-relevant swings. The true breakthrough, the move that would make people say law rather than framing, lives in the multi-step extension of the theorem and the cross-domain prediction, both unproven and hard. If effort were allocated toward the groundbreaking ceiling specifically, the marginal energy belongs on the test-time-compute framing of Section 14.1, because that is where a clean theoretical result meets the question the field is most publicly confused about, and that combination is what occasionally turns a good theorem into a landmark one.

15. Synthesis: one ledger, all instances

The nine instances are best read after the two spines, not before them. The map below is the internal synthesis. It places every project around the shared ledger and is useful for orientation once the signature and companion spines are clear. It is kept here, at the end, rather than at the front, because a reviewer should meet the evidence of each spine first, and because presenting nine instances as one law up front invites the criticism that the ledger is doing the arguing that the results should do.

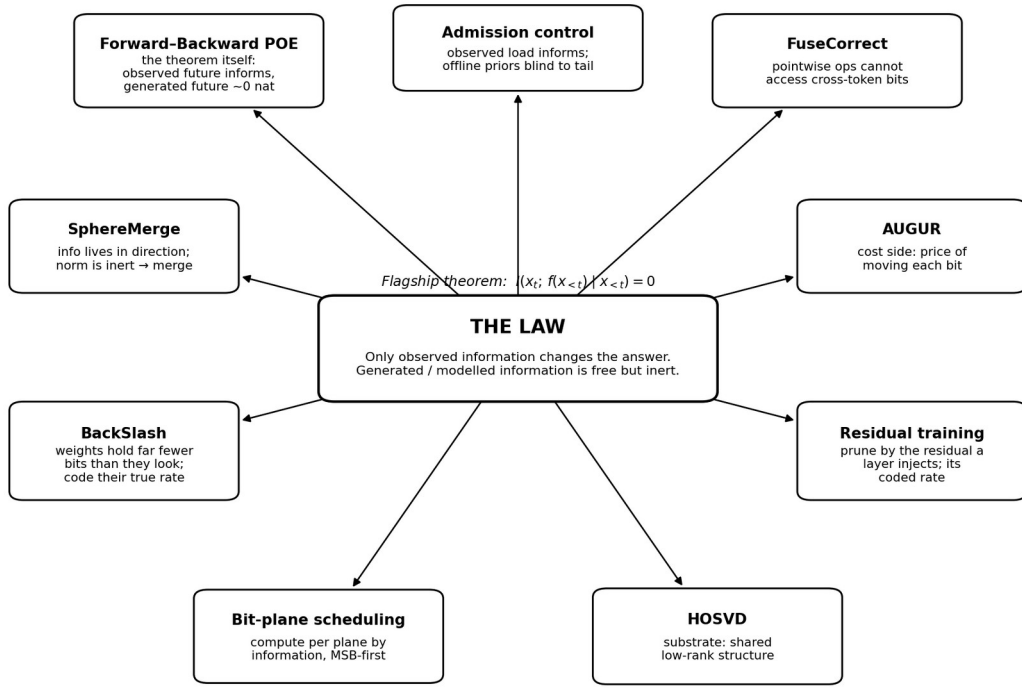


Figure 6. Internal synthesis map: one ledger, nine instances. Every arrow reads as “is an instance of.”

Appendix A: notation

$x_{<t}$ is the observed context up to position t . x_t is the current token. $x_{>t}$ is the observed future. f is an arbitrary deterministic function of the context, that is, a self-generated continuation. p_f and p_b are the forward and backward model distributions. α is the backward exponent in the product-of-experts fusion, default 0.9. $I(\cdot; \cdot | \cdot)$ is conditional mutual information in nats. The oracle ceiling is the information available from the true observed future. E1 to E4 are the four experiments of Section 4.2.

Appendix B: mutual-information estimation protocol

Use one estimator across all domains for comparability. For each evaluation position, compute the forward-only predictive distribution and the fused distribution, which is the forward distribution plus the weighted predictor log-probabilities, then take the KL gain at the observed token. Average over a fixed held-out set and report in nats with a five-seed mean and standard deviation. Always run the row-shuffle null, permuting predictor logits across positions, and confirm the estimate returns to about zero. A non-zero null indicates leakage in the pipeline and invalidates the run. Report the coverage of any predictive intervals against nominal 80, 90, and 95% levels.

Appendix C: reproducibility checklist

Fix and log all seeds for initialization and data order. Record the architecture, parameter count, training steps, and hyperparameters for every model. Store held-out logits and the logits-correlation check for the independence experiments. Version the calibration corpus snapshot for any AUGUR number, and label each value measured, partial, or modelled. Release the serving-loop configuration, meaning the arrival process, rates, and GPU, for the admission experiments. Every headline number should be regenerable from a single logged configuration.

Appendix D: full run sheet for E1–E4

This is the exact sequence a collaborator follows to produce the Figure-2 spine. Estimated wall-clock is 3 to 5 days with the two trainings run in parallel.

Step 1. Train model A, the forward model. Set all seeds, for initialization and data order, to 42; architecture GPT-2 124M (n_embd 768, n_layer 12, n_head 12); data WikiText-103 shuffled with seed 42; 100k steps; save to model_A. Expected time is 1 to 2 days on one GPU.

Step 2. Train model B, the independent predictor. Identical architecture and data, with seeds set to 137 for both initialization and data order; 100k steps; save to model_B. Run it in parallel with Step 1.

Step 3. Verify independence. Compute the correlation of model A and model B held-out logits and require it to fall between 0.70 and 0.90. Above 0.90 the models are too correlated, so re-seed. Below 0.70, investigate under-training or a data mismatch.

Step 4. Run E1. Use model A to predict its own future greedily, fuse with $\alpha = 0.9$, and estimate the mutual information; reproduce ≤ 0.0019 nat. Step 5. Run E2. Use model B as the predictor and record the mutual information. Step 6. Run E3. Use the true observed future $x>t$ and expect about 0.368 nat. Step 7. Run E4. Use the POE-trained backward model and expect about 0.662 nat.

Step 8. Controls. Temperature sweep on E1 at T in {0.0, 0.5, 1.0, 1.5}, expecting about zero throughout. Row-shuffle null on E2, permuting model B logits across positions, expecting the estimate to return to about zero. Optional independent-model scale sweep at 45M, 124M, 355M, and 1.5B. Step 9. Compile. Report E1 to E4 with a five-seed mean and standard deviation, then the two decisive deltas E4 minus E2 and E3 minus E2, and interpret them via the Section 4.2 matrix.

Appendix E: key numbers to reproduce

These are the empirical anchors the program stands on. Treat each as a target to reproduce and, where noted, to extend, and label every one measured, partial, or modelled in the final write-up.

Quantity	Value	Where and status
Self-predicted future (DPI zero)	≤ 0.0019 nat	E1, measured
Observed future (Oracle ceiling)	≈ 0.368 nat	E3, measured
Backward model (POE-trained)	≈ 0.662 nat	E4, measured
Signal within a ≤ 8 -token window	$\approx 73\%$	Forward-Backward, measured
Error-detection AUC, ours vs MLM-PLL	0.704 vs 0.551	Forward-Backward, measured
Backward-advantage retention 124M $\rightarrow$ 1.5B	$\approx 80\%$	Scale invariance, extend to 7B/13B
SphereMerge scoring speedup vs MKA	42 $\times$	SphereMerge, measured
Direction vs norm linear probe	80.06% vs 53.44%	SphereMerge, measured
Massive-activation PC share of variance	81.2%	SphereMerge (LLaMA-3-8B), measured
Pointwise-repair residual floor	$\approx 49\%$	FuseCorrect, measured
Cross-token share of response energy	$\approx 42\%$	FuseCorrect, measured
Admission saturation knee	$\approx 11\text{--}13$ req/s	Admission, reproduce on real vLLM
Offline-gate violation above knee	89–97%	Admission, target for the load-aware gate
BackSlash size reduction	75–90%	BackSlash, measured (up to 90% on Gemma)
GG weight shape parameter	< 2	BackSlash, measured (0.85 to 1.54)
Prunable ratio after BackSlash	80–90%	BackSlash, measured (vs $< 60\%$ normal)
Optimal exp-Golomb parameter	$k = 0$	BackSlash, measured
Residual GGD shape parameter	$\beta < 2$	Residual, measured (super-Gaussian)
Residual structure	depth-heterogeneous,	Residual, measured (DeepSeek)

	batch-stable	
MSB-plane occupancy (GG weights)	$\approx 4\%$	Bit-plane, simulated target
Activation zero-bit fraction (CNNs)	$\approx 92\%$	Bit-plane, prior art (Pragmatic)